Decision Brief: Live Wheat Visualization

Date: 2026-03-11 | **Audience**: Developer learning Wheat | **Phase**: Compiled

Executive Summary

The sprint asked: *How can we build a live visualization of Wheat that teaches the framework by showing itself in action?* The answer is a self-contained HTML replay viewer that time-travels through git history, rendering claims, coverage, and conflicts at each commit. The system works — 156 active claims compiled cleanly, the replay viewer is tested, and the architecture requires zero npm dependencies [d002, p001].

Recommendation

Ship the replay viewer as the primary Wheat demonstration artifact. The architecture is validated: SSE streaming for live development [p001], git-based frame extraction for replay [p014, r046], and hybrid sub-framing for batch commits [x015, p015]. Key next steps:

1. **Enable GitHub Pages** at `/docs` — landing page, replay, brief, and presentation are ready [p023, p024].
2. **Add pedagogical annotations** to key replay frames — phase transitions, first conflict, evidence upgrades [r056, r065, p022]. This was the sprint's biggest gap: the visualization *shows* but doesn't *teach* [r048, r049].
3. **Address accessibility gaps** in the replay viewer — it has had no audit unlike the dashboard [r059]. Five targeted fixes would bring it to WCAG AA [r021].
4. **Add `--watch` mode** to the compiler for a tighter dev loop [r024].

Evidence Summary

Architecture (48 claims, max: tested, status: strong)

The core architecture is a zero-dependency Node.js stack: HTTP server with SSE for live updates [p001], `fs.watch` for file monitoring [r002], and the Wheat compiler as the processing pipeline. The server is 133 lines with no npm dependencies [p001]. The compiler handles 1000 claims in 59ms [e001], though full-loop cost including I/O is higher [x004]. Frame extraction uses `git show <hash>:claims.json` at each commit [r046], with hybrid sub-framing to smooth batch commits [x015, p015]. A key resolved conflict: the prototype

proved the server needed more lines than estimated (133 vs ~80), but remained zero-dependency [p001 > r008].

Pedagogy (23 claims, max: tested, status: strong)

The sprint's most significant finding: the visualization shows *what* happened but never explains *why* [r050]. Only ~5 of the original claims addressed teaching [r048]. The sandbox prototype validates that interactive claim creation teaches Wheat's core mechanics [p025, r068]. Phase-colored timeline framing was chosen over three-act gating [x031, x029] to preserve the sprint's natural interleaving rather than imposing false linearity [x032]. Guided annotations at milestone frames are the recommended approach [r056, r065].

UX Pattern (26 claims, max: tested, status: strong)

AST Explorer's split-pane pattern (claims left, compilation right) is the UI foundation [r004]. The compiler pipeline uses a linear stepper visualization [r005]. Animation timing was fixed at higher playback speeds [x016], empty-delta frames now show metadata context [x017], and milestone markers avoid division-by-zero at single-frame replays [x019]. Claim filtering is recommended at 50+ claims [r027]. Mobile/responsive design is unaddressed [r052].

Accessibility (14 claims, max: tested, status: strong)

The live dashboard was audited and scored "usable but not compliant" [e004]. Key gaps: no ARIA landmarks [r018], no keyboard support on claim rows [r019], SSE updates not announced to screen readers [r020]. Five fixes would achieve WCAG AA [r021]. The replay viewer has had no accessibility audit [r059]. Google Fonts in the explainer template violate self-containment [r022].

Dogfooding (16 claims, max: tested, status: strong)

Self-referential design is validated — the sprint produces its own demo artifact [d001]. Circular dependency risk exists (dashboard value depends on sprint quality) but is accepted [x001]. Selection bias in evidence is acknowledged: most `tested` claims are about the dashboard/compiler because that's what gets prototyped [x010]. Witness corroboration showed narrow diversity — all 15 witnesses supported existing claims, none contradicted [r055, x027].

Quality (11 claims, max: tested, status: strong)

Test suite added in P0 release prep: 8 tests covering happy path, empty claims, malformed JSON, schema validation, conflict detection, burn residue, determinism, and evidence tier sorting. The `--scan` tool enforces self-containment across all HTML artifacts. Success criteria remain subjective [d005] — future sprints should set measurable targets [r037, x007].

Output Format (12 claims, max: tested, status: strong)

Self-contained HTML constraint is enforced [d002, e003]. The server resolves the live-updating-vs-self-contained tension: SSE for development, static HTML for distribution [r007]. Incremental brief updates are recommended for future sprints [r030].

Scope (3 claims, max: documented, status: moderate)

Full creative freedom granted [d004]. Scope expanded significantly beyond the original question — from "build a visualization" to compiler features, accessibility, pedagogy [r039]. This is a natural consequence of the self-referential design.

Audience (3 claims, max: documented, status: moderate)

Target is a single developer seeing Wheat for the first time [d003]. This topic has the thinnest coverage in the sprint — only 2 substantive claims [r051]. No prototype tested actual first-time user comprehension.

Tradeoffs and Risks

Risk	Evidence	Mitigation
Replay viewer has no accessibility audit [r059]	documented	Run same audit as dashboard; apply r021 fixes
Mobile/responsive unaddressed [r052]	documented	Accept for v1; desktop is primary target
Batch commits create uneven replay pacing [x012]	documented	Hybrid sub-framing implemented [x015, p015]
`fs.watch` silently dies on atomic file replacement [e007]	tested	Re-watch on rename event; documented workaround
Self-referential bias in evidence selection [x010]	web	Acknowledged; mitigated by witness diversity
No automated accessibility testing in pipeline [x008]	web	Add axe-core to test suite in future sprint
Unfalsifiable success criterion [x007]	web	Set measurable targets per r037

Resolved Conflicts

1. **p001 vs r008** (architecture): Prototype proved server is 133 lines, not ~80 as estimated. Winner: **p001** (tested > web). The estimate was wrong but the conclusion held — zero dependencies confirmed.
2. **x011 vs r042** (architecture): Challenge found 28% of commits are batch commits with 5+ claims, contradicting the assumption of gradual evolution. Winner: **x011** (documented > web). Led to the hybrid sub-framing solution [x015].

Appendix: Claim Inventory

156 resolved claims across 9 topics. Full inventory in `compilation.json`.

Topic	Claims	Max Evidence	Status	Types
architecture	48	tested	strong	25 factual, 12 risk, 7 rec, 3 feedback, 1 estimate
ux-pattern	26	tested	strong	12 factual, 7 risk, 5 rec, 1 feedback, 1 constraint
pedagogy	23	tested	strong	11 factual, 6 risk, 6 rec
dogfooding	16	tested	strong	8 factual, 5 risk, 2 rec, 1 constraint
accessibility	14	tested	strong	7 factual, 6 risk, 1 rec
output-format	12	tested	strong	7 factual, 2 risk, 1 rec, 1 constraint, 1 feedback
quality	11	tested	strong	4 factual, 3 feedback, 2 risk, 1 rec, 1 constraint
audience	3	documented	moderate	1 constraint, 1 factual, 1 risk
scope	3	documented	moderate	1 constraint, 1 estimate, 1 risk

Compilation certificate: sha256:38b4f2a0e57f4d9da95260adedfd6316fc59c85cacc07cf69d78cb83ae0551c5 | Compiler: wheat v0.2.0 | Claims: 156 resolved / 165 total | Compiled: 2026-03-11T12:23:33.337Z